Backend · revision notes

FastAPI

Type-hinted APIs, Pydantic, dependency injection, async, auth, testing

1 What FastAPI is

Definition

FastAPI is a modern Python web framework for building APIs. It uses type hints for validation and automatically generates **OpenAPI docs** at `/docs`. It runs on **ASGI** (Starlette + Pydantic) and supports `async`.

Think of FastAPI like a smart receptionist

1

Caller

Client request

Sends JSON to /items/5

2

Checks the form

Pydantic validation

Rejects bad data with a clear 422 error

3

Right department

Path operation function

Your code does the real work

4

Reply letter

Response model

Only the allowed fields go back as JSON

The smallest API (run: uvicorn main:app --reload)

```python
from fastapi import FastAPI

app = FastAPI(title="Shop API")

@app.get("/")
def root():
    return {"status": "ok"}

@app.get("/items/{item_id}")
def read_item(item_id: int, q: str | None = None):    # path + query params come from the signature
    return {"item_id": item_id, "q": q}
```

- Interactive docs: Swagger UI at `/docs`, ReDoc at `/redoc`, raw schema at `/openapi.json`.
- `item_id: int` converts and validates the path parameter; non-numbers return **422** automatically.
- Run with **uvicorn** (dev: `--reload`; production: several workers or gunicorn with uvicorn workers).

2 Request bodies and Pydantic models

Validate input, shape output

```python
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, EmailStr, Field

class UserIn(BaseModel):
    name: str = Field(min_length=1, max_length=80)
    email: EmailStr
    password: str = Field(min_length=8)

class UserOut(BaseModel):          # never includes the password
    id: int
    name: str
    email: EmailStr

app = FastAPI()
DB: dict[int, dict] = {}

@app.post("/users", response_model=UserOut,
status_code=status.HTTP_201_CREATED)
def create_user(user: UserIn):
    if any(u["email"] == user.email for u in DB.values()):
        raise HTTPException(status_code=409, detail="Email already
registered")
    uid = len(DB) + 1
    DB[uid] = {"id": uid, **user.model_dump()}
    return DB[uid]                 # filtered through UserOut
```

- A **Pydantic model** parameter is read from the JSON body; scalar parameters come from the path/query. Use `Query()`, `Path()`, `Body()` and `Header()` to add rules.
- `response_model` filters and documents the output (hide passwords and internal fields).
- Raise `HTTPException(status_code, detail)` for expected errors; add `exception_handler`s for custom ones.
- Pydantic v2 uses `model_dump()` / `model_validate()`; `from_attributes=True` lets it read ORM objects.

3 Dependency injection and routers

Depends: shared logic, DB sessions, auth

```python
from typing import Annotated
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session

def get_db():
    db = SessionLocal()
    try:
        yield db                # code after yield runs when the request ends
    finally:
        db.close()

def get_current_user(token: Annotated[str, Depends(oauth2_scheme)], db: Session
= Depends(get_db)):
    user = decode_and_load(token, db)
    if not user:
        raise HTTPException(status_code=401, detail="Invalid token")
    return user

router = APIRouter(prefix="/items", tags=["items"])

@router.get("/mine")
def my_items(user = Depends(get_current_user), db: Session = Depends(get_db)):
    return db.query(Item).filter(Item.owner_id == user.id).all()

# app.include_router(router)
```

- `Depends()` is FastAPI's **dependency injection**: reusable pieces (DB session, current user, pagination) are resolved per request and are easy to override in tests.
- **APIRouter** splits the app by feature (`/users`, `/items`) with their own prefixes, tags and shared dependencies.
- Settings: use `pydantic-settings` (`BaseSettings`) to read configuration from environment variables.
- **Background tasks** (`BackgroundTasks`) are for small post-response work; use Celery/RQ/arq for heavy jobs.

4 async def vs def, auth and middleware

You write	FastAPI runs it	Use when
`async def`	on the event loop	you `await` async libraries (httpx, asyncpg, async SQLAlchemy)
plain `def`	in a thread pool	you call blocking code (requests, sync DB drivers)

Common mistake

Blocking calls (time.sleep, requests.get, a sync DB query) inside `async def` freeze every request. Either make the function plain `def` or use an async library.

- **Auth**: OAuth2 password flow with JWT access tokens (`OAuth2PasswordBearer`), hash passwords with bcrypt/argon2 (never store them plain).
- **CORS**: `app.add_middleware(CORSMiddleware, allow_origins=[...])` with an explicit origin list.
- **WebSockets** (`@app.websocket`) and **Server-Sent Events** support real-time features.
- **Lifespan** handlers open and close shared resources (DB pools) at startup and shutdown.
- **Testing**: `TestClient(app)` (or `httpx.AsyncClient`) plus `app.dependency_overrides` to swap the DB or the user.

Testing with TestClient

```
from fastapi.testclient import TestClient

client = TestClient(app)

def test_create_user():
    r = client.post("/users", json={"name": "Asha", "email": "a@b.com",
"password": "secret123"})
    assert r.status_code == 201
    assert "password" not in r.json()
```

Revision

Type hints → validation + docs, Pydantic in/out models, response_model hides fields, Depends for DB/auth, async def only with async libraries, APIRouter per feature, JWT + hashed passwords, explicit CORS, TestClient + dependency_overrides, uvicorn workers in production.

5 Rapid-fire Q&A

Q. Why is FastAPI fast?

A. It is built on ASGI (Starlette) with async support and Pydantic validation, so one process serves many concurrent I/O-bound requests efficiently.

Q. How does FastAPI validate data?

A. Through Python type hints and Pydantic models. Invalid input returns a 422 response listing which fields failed.

Q. What are the automatic docs?

A. FastAPI generates an OpenAPI schema from your code and serves Swagger UI at /docs and ReDoc at /redoc.

Q. async def vs def endpoints?

A. async def runs on the event loop and must only await async code; plain def runs in a thread pool so blocking code does not stall the server.

Q. What is dependency injection in FastAPI?

A. Depends() declares reusable functions (DB session, current user, pagination) that FastAPI resolves per request, making code shorter and easily testable via dependency_overrides.

Q. What is response_model for?

A. It validates and filters the returned data to a Pydantic model, so fields like passwords never leak, and documents the output.

Q. FastAPI vs Django vs Flask?

A. FastAPI: modern async APIs with validation and docs. Django: full-stack with ORM, admin and auth built in. Flask: minimal, bring your own pieces.

Q. How do you handle authentication?

A. OAuth2 password flow with JWT tokens via OAuth2PasswordBearer and a get_current_user dependency; passwords are hashed with bcrypt/argon2.

Q. How do you run FastAPI in production?

A. uvicorn (or gunicorn with uvicorn workers) behind a reverse proxy like Nginx, in Docker, with settings from environment variables.

Q. How do you test a FastAPI app?

A. Use TestClient (or httpx.AsyncClient) with pytest, and override dependencies to use a test database or fake user.

Q. What is Pydantic?

A. A data validation library that parses and validates data into typed models; v2 uses model_validate and model_dump.

Q. Path vs query vs body parameters?

A. Path params are in the URL template ({id}), query params are other simple arguments (?q=), and the body is a Pydantic model sent as JSON.